

LAB ACTIVITY (Exp-06&07)

1. Implement Quick sort algorithm. Find execution time for already sorted order input and unsorted input.

```
#include <stdio.h>

// Function to swap two numbers
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Partition function
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // last element as pivot
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

// QuickSort function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
}
```

```

}
printf("\n");

return 0;
}

```

```

PS D:\SEMESTER I\LAB WORK\Data structures and algorithm> cd "d:\SEMESTER I\LAB WORK\Data structures and algorithm\lab7(0110)" ; if ($?) { gcc quicksort.c -o quicksort }
Enter number of elements: 8
Enter 8 elements: 12 4 2 11 9 10 5 4
Sorted array: 2 4 4 5 9 10 11 12
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab7(0110)> cd "d:\SEMESTER I\LAB WORK\Data structures and algorithm\lab7(0110)" ; if ($?) { gcc quicksort.c -o quicksort }

```

execution time for already sorted order input:- $O(n^2)$

execution time for unsorted input:- $O(n \log n)$

2. Implement Linear search algorithm.

```

#include <stdio.h>

int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key)
            return i;
    }
    return -1;
}

int main() {
    int n, key;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &key);

    int result = linearSearch(arr, n, key);
    if (result != -1)
        printf("Element %d found at index %d\n", key, result);
    else
        printf("Element not found\n");

    return 0;
}

```

```

PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab7
tructures and algorithm\lab7(0110)\ " ; if ($?) { gcc linear_
\linear_search }
Enter number of elements: 7
Enter 7 elements: 10 5 7 2 9 11 3
Enter element to search: 7
Element 7 found at index 2
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab7

```

3. Implement binary search algorithm.

```

#include <stdio.h>

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main() {
    int n, key;
    printf("Enter number of elements (sorted order): ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d sorted elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &key);

    int result = binarySearch(arr, n, key);
    if (result != -1)
        printf("Element %d found at index %d\n", key, result);
    else
        printf("Element not found\n");

    return 0;
}

```

Element 7 found at index 2

```
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab7(0110)>
structures and algorithm\lab7(0110)\\" ; if ($?) { gcc binary_search.c
\\binary_search }
```

Enter number of elements (sorted order): 8

Enter 8 sorted elements: 2 4 8 10 45 78 98 102

Enter element to search: 98

Element 98 found at index 6

```
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab7(0110)>
```